SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 6

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 6: Lists of shows Arrays, objects, loops

A real app doesn't show one show: it shows lists. Today you learn how JavaScript keeps many values together (**arrays**), how it describes one thing with named values (**objects**), and how to go through a whole list with a **loop**.

The file **data.js** (a read-only tab above the code box) holds twelve shows, each one an object with a name, genres, a rating and more. You start in the Console, exploring the data. Then you write small functions that pick shows from the list: the best ones, one by its id, the ones that match a search, the ones in a genre.

As before, a helper file, **page.js**, uses your functions to fill in the page. It also scrolls the preview to the newest part you've made.

Step 1. A list: arrays ~15 min

Teaches: Arrays: `[a, b, c]`, `list[0]` is the first item, `.length`, `.push()`.

TASK

1. Create `favourites`: an array with the names of three shows you like.
2. Log the first one, and log how many there are.
3. Add a fourth show with `push`. Then log the whole array.

COMMON MISTAKES

- `favourites[1]` for the first item (counting starts at 0).
- Forgetting the commas between items.
- Round brackets instead of square ones: `("Dark", "Fleabag")`.

A CORRECT ANSWER

```
// app.js: lists of shows.
// data.js (a read-only tab) gives you `shows`: an array of 12 show objects.

const favourites = ["Dark", "Fleabag", "The Wire"];
console.log(favourites[0]);
console.log(favourites.length);

favourites.push("Chernobyl");
console.log(favourites);
```

Step 2. One show: objects ~15 min

Teaches: Objects: `{ key: value, ... }`. Read with `obj.key`, change with `obj.key = value`.

TASK

1. Create an object `myShow` for a show you like, with three properties: `name` (text), `year` (the year it started, a number) and `rating` (a number).
2. Log its name.
3. Change the rating to your own score, with a dot and `=`.
4. Log the whole object.

COMMON MISTAKES

- `=` instead of `:` inside the object: `name = "Fleabag"`.
- A missing comma between properties.
- Quotes around numbers: `year: "2016"`.

A CORRECT ANSWER

```
// ... unchanged ...
favourites.push("Chernobyl");
console.log(favourites);

const myShow = {
  name: "Fleabag",
  year: 2016,
  rating: 8.4,
};
console.log(myShow.name);

myShow.rating = 10;
console.log(myShow);
```

Step 3. The twelve shows ~15 min

Teaches: Arrays of objects: `shows[0].name`. The last item is `list[list.length - 1]`.

TASK

Log four things about the data:

1. how many shows there are
2. the name of the first show
3. the rating of the second show
4. the name of the last show, using `shows.length - 1`

COMMON MISTAKES

- `shows[12]` for the last show (it's `shows[11]`).
- `shows.name` (the array has no name; each show has).

NOTE

Encourage students to open the data.js tab and read one show object in full before they start.

A CORRECT ANSWER

```
// ... unchanged ...
myShow.rating = 10;
console.log(myShow);

console.log(shows.length);
console.log(shows[0].name);
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);
```

Step 4. Go through every show ~15 min

Teaches: `for (const item of list) { ... }` runs the block once per item.

TASK

Write a `for...of` loop through `shows` that logs the name of every show. You'll see twelve new lines in the Console.

If you like, log the rating too: `console.log(show.name, show.rating);`.

COMMON MISTAKES

- `shows.name` inside the loop instead of `show.name`.
- `for (const show in shows)`: `in` gives the index numbers, not the shows.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);

for (const show of shows) {
  console.log(show.name);
}
```

Step 5. Keep only the best ~20 min

Teaches: Pick items: start with `[]`, loop, and `push` the ones that pass an `if`.

TASK

Replace the `TODO` comment: if a show's rating is **9 or more**, push the show into `best`.

The Console shows `best.length`. How many shows are rated 9 or more?

Then try it with `8.8` instead of `9`, and change it back.

COMMON MISTAKES

- The `push` outside the `if` (every show is added).
- `const best = [];` inside the loop (it starts empty every round).
- `>` instead of `>=` (the 9.0 shows are left out).

A CORRECT ANSWER

```
// ... unchanged ...
  console.log(show.name);
}

const best = [];
for (const show of shows) {
  if (show.rating >= 9) {
    best.push(show);
  }
}
console.log(best.length);
```

Step 6. filter: the short way ~20 min

Teaches: `list.filter((item) => test)` gives a new array with the items that pass the test.

TASK

Under your code, write `topRated(list)`. It returns the shows in `list` with a rating of **9 or more**, using `filter`.

The Top rated section on the page fills with five cards.

COMMON MISTAKES

- Forgetting `return` before `list.filter`.
- Using `shows` inside the function instead of the parameter `list`.
- Curly brackets after the arrow without a `return`: `(show) => { show.rating >= 9 }` keeps nothing.

A CORRECT ANSWER

```
// ... unchanged ...
}
console.log(best.length);

function topRated(list) {
  return list.filter((show) => show.rating >= 9);
}
```

Step 7. map: one thing from each ~15 min

Teaches: `list.map((item) => newValue)` gives a new array with one result per item.

TASK

1. Write `names(list)` that returns an array with the name of every show in `list`, using `map`.
2. Log `names(shows)` to see the result in the Console.

COMMON MISTAKES

- Using `filter` instead of `map` (you get whole shows back).
- `shows.name` instead of `show.name` inside the arrow function.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.rating >= 9);
}

function names(list) {
  return list.map((show) => show.name);
}

console.log(names(shows));
```

Step 8. find: one show by its id ~15 min

Teaches: `list.find((item) => test)` gives back the first matching item, or `undefined`.

TASK

1. Write `getById(list, id)` that returns the show in `list` with that `id`, using `find`.
2. Test it: `console.log(getById(shows, 2).name);` should print Breaking Bad. (Without `.name`, you'd see the whole show object.)

page.js uses it for **Pick of the day**: `getById(shows, 7)`.

COMMON MISTAKES

- `show.id = id` (one equals sign) always gives the first show.
- Using `filter` and getting a list with one show in it.
- `getById(shows, "2")`: the id in data.js is a number, and `===` checks the type.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(names(shows));

function getById(list, id) {
  return list.find((show) => show.id === id);
}

console.log(getById(shows, 2).name);
```

Step 9. Search by name ~20 min

Teaches: Search: `trim()`, `toLowerCase()` on both sides, then `includes()`.

TASK

We added the start of `searchByName(list, term)`. Replace the two `TODO` comments:

1. Make `needle`: the `term`, trimmed and in lower case.
2. Return the shows in `list` whose name, in lower case, includes `needle`.

Then type in the search box on the page. Try `office`, `bad` and `THE`. Why does How I Met Your Mother appear for `THE`?

COMMON MISTAKES

- Making only the search word lower case, not the show's name.
- `show.name === needle` instead of `includes` (only exact names match).
- `toLowerCase` without brackets.

NOTE

On Day 7 students write the event code that page.js does here (listening to the search box). Today, the point is the function.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(getById(shows, 2).name);

function searchByName(list, term) {
  const needle = term.trim().toLowerCase();
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}
```

Step 10. Browse by genre ~15 min

Teaches: `array.includes(value)` checks whether a list contains a value.

TASK

Write `getByGenre(list, genre)`. It returns the shows in `list` whose `genres` include `genre`.

Then click the genre buttons on the page. What happens with **Western**?

COMMON MISTAKES

- `show.genre` instead of `show.genres` (the key has an s).
- `show.genres === genre` (an array is never equal to one string).

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}

function getByGenre(list, genre) {
  return list.filter((show) => show.genres.includes(genre));
}
```

Extra 1. Best first ~20 min

Teaches: `[...list].sort((a, b) => b.x - a.x)` sorts a copy, biggest first.

TASK

Write `sortByRating(list)` that returns a **copy** of the list, highest rating first. A new section, **Best first**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function sortByRating(list) {
  return [...list].sort((a, b) => b.rating - a.rating);
}
```

Extra 2. Count the genres ~20 min

Teaches: `obj[key]` reads or sets a property whose name is in a variable.

TASK

Write `countByGenre(list)` that returns an object like `{ Comedy: 4, Drama: 8, ... }`. Use loops and an object, not a new array method.

When it works, the genre buttons on the page show their counts.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function countByGenre(list) {
  const counts = {};
  for (const show of list) {
    for (const genre of show.genres) {
      if (counts[genre]) {
        counts[genre] = counts[genre] + 1;
      } else {
        counts[genre] = 1;
      }
    }
  }
  return counts;
}
```

Extra 3. Watch everything ~15 min

Teaches: Add up a list: `let total = 0;` before the loop, `total = total + ...` inside.

TASK

Write `totalWatchTime(list)` that returns the total minutes needed to watch every episode of every show in the list (each show: episodes times runtime).

A new section, **Watch everything**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function totalWatchTime(list) {
  let total = 0;
  for (const show of list) {
    total = total + show.episodes * show.runtime;
  }
  return total;
}
```